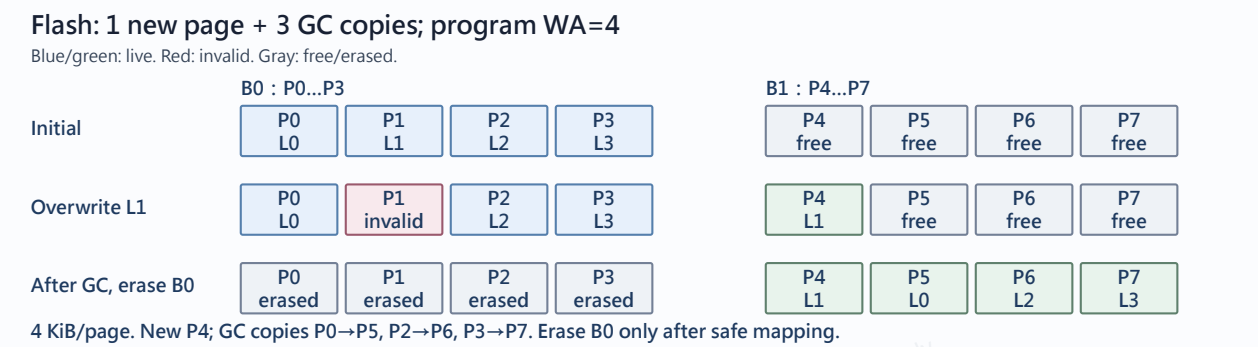
Media mechanisms and metric denominators

HDDs use magnetic surfaces, rotating platters and moving heads. SSDs use nonvolatile semiconductor media with controllers/FTLs. Tape supports streaming backup/archive with long positioning costs. Hosts address logical blocks rather than controlling every physical location. SSD costs cannot be modeled as head seek plus rotation.

Metric	Denominator/boundary	Distinct quantity
Bandwidth	Completed bytes/elapsed seconds	Interface/media peak need not equal end-to-end
IOPS	Completed requests/second	Different request sizes have different byte rates
Service time	Actual device service interval	Excludes time not yet selected
Response latency	Arrival→specified completion	Can include queue, scheduling, copy
Durability	Failures tolerated after acknowledged success	Completion may still be volatile

Flash out-of-place writes and GC



Fix 4 KiB NAND pages and four-page 16 KiB erase blocks. B0 P0/P1/P2/P3 hold L0/L1/L2/L3; B1 P4…P7 are erased. Overwriting L1 programs new P4, sets L1→P4, and invalidates P1. An unerased NAND page cannot be arbitrarily overwritten in place.

GC copies live L0:P0→P5, L2:P2→P6, L3:P3→P7. Only after live contents and necessary mapping are safe can it erase B0. One host page causes 1 new+3 copied physical page programs, write amplification 4 in this crash-free model. If L2 is discarded first, only 2 copies remain,3 programs and WA=3. Mapping writes and other controller overhead are excluded.

Flash out-of-place writes and GC (continued)

Discard/TRIM says logical data is no longer needed; it does not promise immediate erase or secure deletion. Wear leveling distributes erase load. GC, over-provisioning, and controller caches change latency. Power-loss correctness additionally needs recoverable mapping/data commit order; a normal-path diagram does not prove it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometry, LBA and access costs

Tracks at the same radius form a cylinder. Seek changes radius, rotation brings a sector under the head, and transfer moves bytes. Zones and remapping alter geometry. LBA is a one-dimensional logical index, not a permanent physical cylinder; SSDs have no head radius for SSTF to optimize.

With 512-byte logical sectors, LBA 4096 starts at 2 MiB. With 4 KiB blocks, the same index starts at 16 MiB. Unsigned 32-bit LBA addresses $2^{32} \times 512 = 2^{41}$ bytes=2 TiB; this is an address-space calculation, not a universal partition/API limit.

A512-byte upper write on a4 KiB physical/atomic unit may need read-modify-write or device translation. Filesystem block size does not establish atomic sector size; alignment and crash granularity follow device/driver contracts. Bad-sector remapping can preserve the LBA while adding latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disk capacity: expand the geometry and units

Assume a disk assembly has 10 double-sided platters, with the top and bottom outer surfaces unused. There are `10×2−2=18` usable surfaces. Each surface has 2,048 tracks, each track 4,096 sectors, and each sector 8,192 bytes. Geometry is uniform and no space is deducted for spare sectors or formatting.

$$18 \times 2{,}048 \times 4{,}096 \times 8{,}192 = 1{,}236{,}950{,}581{,}248 \text{ bytes}.$$

Dividing this same quantity by `2³⁰` gives **1,152 GiB**; by `2⁴⁰` it gives **1.125 TiB**. In decimal units, `1 GB = 10⁹ bytes`, so it is approximately **1,236.95 GB**. State the units when comparing values; this is the capacity under the specified geometry, not a guarantee of formatted usable space.

1000 completed 4 KiB requests/s gives 3.90625 MiB/s; 1000 of64 KiB gives 62.5 MiB/s. The same IOPS does not mean the same bandwidth, and this arithmetic is not a product measurement. Comparisons fix request size, random/sequential pattern, R/W mix, queue depth, cache, and synchronization contract.

Derive timing from rotational speed and track size

Assume 6,000 rpm, 1,024 sectors per track, and 4 KiB per sector. Adjacent sectors are continuously readable; ignore sector gaps, controller, cache, and queueing costs.

Step	Calculation	Result
Revolutions per second	6,000 ÷ 60	100 rev/s
Revolution time	1 ÷ 100	0.01 s = 10 ms
Data per revolution	1,024 × 4 KiB	4 MiB
Ideal sequential rate	100 × 4 MiB/s	400 MiB/s
Average rotational wait	Uniform target phase, half a revolution	5 ms

With a fixed 3-second seek, reading 4 MiB laid out continuously on one track takes one revolution after the target sector arrives: `4 MiB ÷ 400 MiB/s = 0.01 s`. Average service time is `3 + 0.005 + 0.01 = 3.015 s`. This is a specified model, not a measured device speed; real geometry, gaps, caches, and overhead can change the result.

A7200 RPM,64 KiB read

Fix seek 5 ms, uniform rotational phase,7200 RPM, transfer 128 MiB/s, request 64 KiB, no cache/controller overhead. A rotation is60,000/7200=25/3 ms; mean rotation 25/6 ms; transfer 64 KiB/(128 MiB/s)=125/256 ms.

$$service = 5 + 25/6 + 125/256 = 7415/768 \approx 9.65495 \text{ ms}.$$

With queue wait 15 ms, response≈24.65495 ms. One-at-a-time service without overlap gives 1000/9.65495≈103.574 IOPS, not a multiqueue/cache/adjacent-track prediction. Sequential 1 GiB has transfer-only lower bound 1024/128=8 s, with positioning/queue added. Random small-I/O positioning dominates differently from streaming bandwidth.

Host, NAS and SAN responsibilities

Path	Client abstraction	Filesystem metadata owner	Additional failure boundary
Host-attached block	Local driver/controller blocks	Local host filesystem	Controller/device
NAS	Remote file/directory operations	File server	Network, server, cache/retries
SAN block	Logical blocks over storage fabric	Host using the LUN	Fabric, target, multipath

For a4 KiB NAS read, the client sends handle/offset/length, the server resolves filesystem metadata and issues block I/O. For SAN, the local filesystem first maps offset to LBA, then block transport reaches the target. The same array can sit below both while pathname/cache/retry responsibility differs.

A SAN LUN shared by two hosts does not provide filesystem distributed locking/coherence. Concurrently mounting an ordinary exclusive-host filesystem for writes can corrupt metadata. A remote mount success proves neither all future I/O availability nor durable completion.

Host, NAS and SAN responsibilities (continued)

Paths A/B to one target can survive A's link loss while B/target remain; target loss defeats both. Two paths are not two independent data copies. Retries preserve request identity/completion semantics; an unknown prior write commit cannot authorize blindly repeating nonidempotent upper operations. Latency depends on protocol, size, network, cache and queues, not the attachment name alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Scheduling on one fixed batch

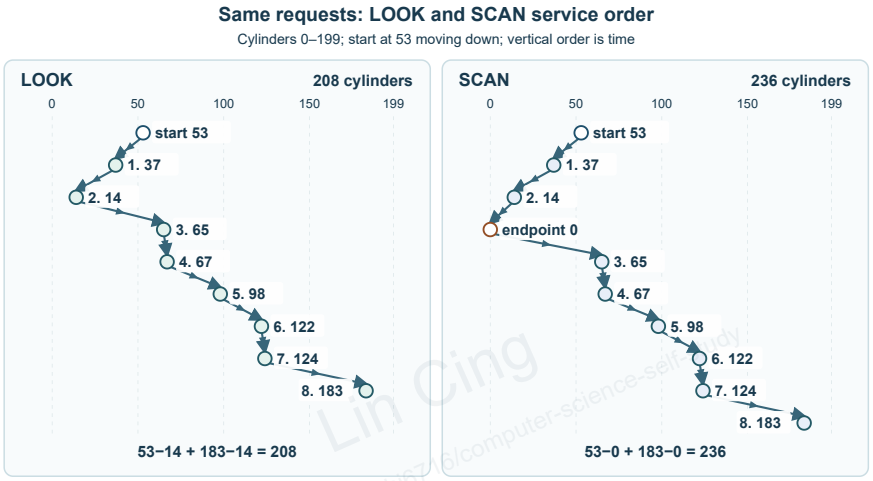
FCFS follows arrivals; SSTF chooses nearest pending cylinder; SCAN serves one direction to a physical endpoint then reverses; C-SCAN wraps and serves only one direction. LOOK reverses at the last request; C-LOOK wraps between extreme requests.

Fix cylinders 0…199, head 53, all arrivals 0, requests 98,183,37,122,14,124,65,67. Cost is absolute head movement until the last request completes, including turn/wrap, excluding rotation, transfer and final empty travel. SSTF ties choose smaller cylinder.

Policy	Path including turn/wrap	Movement
FCFS	53→98→183→37→122→14→124→65→67	640
SSTF	53→65→67→37→14→98→122→124→183	236
LOOK down	53→37→14→65→67→98→122→124→183	208
SCAN down	53→37→14→0→65→67→98→122→124→183	236
C-SCAN up	53→65→67→98→122→124→183→199→0→14→37	382
C-LOOK up	53→65→67→98→122→124→183→14→37	322

Circular returns count physical movement but serve no requests during wrap. A problem excluding that cost must explicitly use another accounting model.

Scheduling on one fixed batch (continued)



0 is a physical endpoint, not a request. Both paths stop after serving request 183.

LOOK=|53−14|+|183−14|=39+169=208. SCAN=|53−0|+|183−0|=53+183=236. SSTF's236 is coincidental, not the same path. FCFS differences 45+85+146+85+108+110+59+2=640; SSTF 12+2+30+23+84+24+2+59=236. LOOK 208 disproves SSTF's global-batch optimality.

The same requests with an upward initial direction

Use cylinders 0–199, start at 53, and note the previous position was 50. This example explicitly starts toward larger cylinder numbers. All eight requests are pending initially: `98, 183, 37, 122, 14, 124, 65, 67`. Charge absolute movement for every leg and stop after the last request; rotation and transfer are excluded.

Policy	Route (199 is a turn point, not a request)	Leg distances	Total
Upward SCAN	53→65→67→98→122→124→183→199→37→14	12, 2, 31, 24, 2, 59, 16, 162, 23	331
Upward LOOK	53→65→67→98→122→124→183→37→14	12, 2, 31, 24, 2, 59, 146, 23	299

Under this convention, SCAN reverses at the physical endpoint 199; LOOK reverses at the last request in that direction, 183. Neither route wraps around. With the **downward** initial direction in the other worked example, SCAN and LOOK instead total 236 and 208. The request batch alone does not determine the number: state the direction, endpoint rule, and stopping condition.

Arrival assumptions and waiting guarantees

A finite batch has no infinite starvation. SSTF's starvation risk needs continuing nearby arrivals that indefinitely defer a distant request. SCAN/LOOK reduces that bias, but finite wait needs bounded range, per-sweep service and admission/track rules; an unbounded stream at one position cannot silently be served in finite time.

Controller reordering, merging and parallel queues alter actual device behavior. SSD erase/parallelism differs from seek geometry. Movement/request is an average positioning proxy, not actual response time. Deadlines, fairness and throughput require their own cost model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disk layout and management

Physical formatting organizes media sectors/error controls, often by device manufacturing/management. Partitioning divides logical address ranges. Logical formatting creates filesystem metadata/free-space structures. Mount attaches a recognized filesystem to a namespace. Formatting is neither secure erase nor automatic mount.

For a teaching 1024 MiB disk with 512-byte sectors, reserve[0,1) MiB for abstract boot/partition metadata. A=[1,101) MiB, B=[101,301) MiB.

Partition	First LBA	Sector count	Exclusive end
A100 MiB	2048	204800	206848
B200 MiB	206848	409600	616448

Ranges do not overlap. This is not a complete GPT specification. B mounting does not establish A's formatting/data integrity. A common mountpoint does not prove device/UUID identity. Bootloader kernel discovery and kernel root-mount need early driver dependencies from Chapter 2.

Disk layout and management (continued)

Controller remapping keeps an LBA while moving a defective physical sector to a spare. It cannot reconstruct already unreadable bytes by itself. Checksums/redundancy can detect/rebuild only with trustworthy data and error propagation. Boot-block damage, filesystem-metadata damage, and data-block damage require different recovery objects/versions. A partition existing is not proof that the disk is repaired. Scrubbing competes with foreground I/O and depends on spare/detection capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Swap backing and slot lifetime

Anonymous pages may need swap; clean file pages can be reread from their file. Swap partition/file are allocation and access-path choices. Prebuilt extent maps can reduce normal filesystem lookup, so an old swap-file overhead is not universal. There is no unconditional twice-RAM sizing formula.

Fix 4 KiB pages, slot S7/value 41, and two private-COW logical pages backed by it. Count only surviving swap-PTE references, not Linux swap_map encodings or all swapcache holds.

Event	P1	P2	S7 swap-PTE refs
Initial	Swap S7	Swap S7	2
P1 reads/private-writes 42	Resident F23/42	Swap S7, expects 41	1
P2 reads back	F23/42	Resident F24/41	0
No I/O/cache ownership remains	Resident	Resident	Only now reclaim S7

P1's new value cannot overwrite S7 while P2 needs 41; allocate new backing if required. This does not fix Linux fault counts or swapcache-sharing choices. Existing swap PTEs, I/O or cache holds prevent early slot reuse after one process exits. Counts need explicit scope/increment/decrement rules. Swap extends current execution's memory, not backup; hibernation has a different recovery contract. More swap can help commit but does not fix physical working-set thrashing.

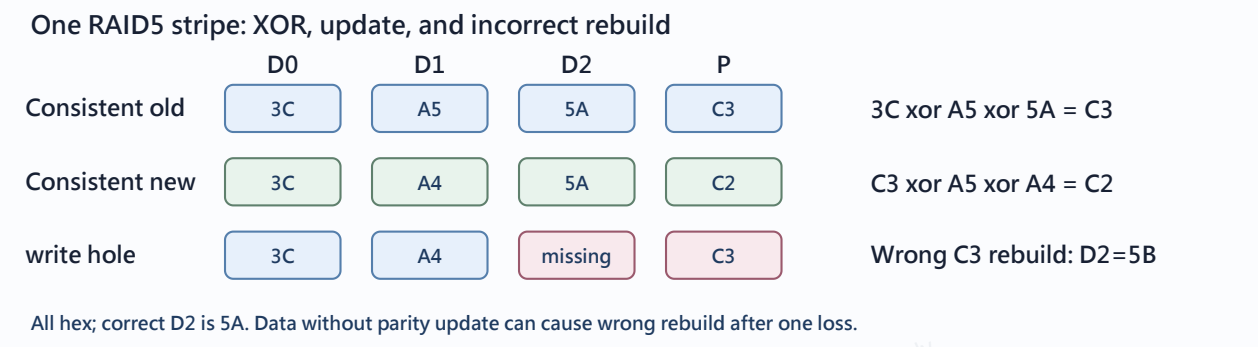
RAID capacities and device-loss assumptions

For N equal disks of capacity D, excluding metadata and spares:

Level	Layout	Capacity	Disk-loss guarantee
0	Stripe	ND	No redundancy
1	Two-copy pairs	D per pair	One loss per pair
2	Bit striping, ECC	Code-dependent	Depends on ECC
3	Fine stripe, dedicated parity	(N−1)D	One disk
4	Block stripe, dedicated parity	(N−1)D	One disk
5	Block stripe, rotating parity	(N−1)D	Any one disk
6	Two independent parity relations	(N−2)D	Any two, with suitable code

Two two-disk mirrors use four disks and yield 2D; one four-copy mirror yields D. Rebuild adds load. RAID cannot replace backups against deletion or common failures.

RAID5 XOR and the write hole



One stripe has hex D0=3C, D1=A5, D2=5A, P=3C xor A5 xor 5A=C3. Missing D1 reconstructs C3 xor 3C xor 5A=A5. Two unknown data values cannot be uniquely solved from one XOR relation.

Change D1 to A4. New parity C3 xor A5 xor A4=C2. Cold uncached read-modify-write reads old D1 and P, writes new D1/P',2 reads+2 writes excluding log/cache. A full three-data stripe can calculate parity directly and do 4 writes without old-stripe reads.

If A4 is durable but parity remains C3, all data disks can still be directly read but the relation is inconsistent. Then losing D2 and rebuilding with old parity yields C3 xor 3C xor A4=5B, not 5A. Single-device tolerance assumes coherent redundancy; it does not include arbitrary interrupted parity update plus a later loss.

RAID5 XOR and the write hole (continued)

Properly protected write-back cache, journaling/parity protocols or COW can protect that transition under implemented commit/recovery rules. Mirrors, checksums or a RAM flag alone do not solve it. Redundancy and persistence are separate contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why RAID6 needs an independent second equation

Over a suitable binary field, P=ΣDi, Q=ΣαiDi. Missing locations a/b have distinct coefficients. Subtract known values to get A+B=Pmissing and αaA+αbB=Qmissing. Substitute A=Pmissing+B:

$$(\alpha_a + \alpha_b)B = Q_{missing} + \alpha_a P_{missing}.$$

Distinct coefficients make their sum nonzero/invertible, so B then A is unique. Duplicating the same XOR gives dependent equations and cannot solve two unknowns. Codes specify field polynomial, layout, and error/erasure model.

Use GF(2⁸), modulus 0x11D, coefficients 1/2/4, data 3C/A5/5A. Q=3C xor(2 • A5) xor(4 • 5A)=3C xor 57 xor 75=1E. Lose D1/D2: Pmissing=FF, Qmissing=22. Then 6B=22 xor(2 • FF)=22 xor E3=C1. Inverse 6=7A; B=7A • C1=5A, A=FF xor 5A=A5. Multiplication is carryless with field reduction, not integer multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RAID 6: check each pair of erasures

Fix one stripe with each symbol on a distinct disk. Its data symbols are D_i, $P=\oplus D_i$ and $Q=\oplus(\alpha_i D_i)$. Arithmetic is in $GF(2^8)$, where addition is XOR. The coefficients α_i are **nonzero and pairwise distinct**. Assume the failed locations are known, every surviving symbol is correct, and at most two symbols in this stripe are erased.

Erased symbols	Information and reconstruction
D_a, D_β	Subtract known data from P and Q; the two-equation determinant $\alpha_a \oplus \alpha_\beta$ is nonzero, so the solution is unique.
P, D_a	Subtract the other data contributions from Q to obtain $\alpha_a D_a$; multiply by α_a^{-1}, then recompute P.
Q, D_a	Subtract the other data from P to obtain D_a, then recompute Q.
P, Q	All data survive, so recompute both parity symbols.

Nonzero coefficients make the P-plus-data case recoverable through Q; distinct coefficients make the two-data equations independent. This is a known-location erasure guarantee. It does not by itself identify arbitrary silent corruption or cover common-cause failures outside the model.

Recover D_1 when P and D_1 are erased

Use $GF(2^8)$ with reduction polynomial `0x11D`; all values below are hexadecimal bytes. Let D_0=`3C`, D_1=`A5`, D_2=`5A`, with coefficients `01, 02, 04`. Initially `P=C3` and `Q=1E`.

P and D_1 are erased, while D_0, D_2 and Q survive. Subtract the known contributions from Q; subtraction is XOR in this field:

$$02D_1 = Q \oplus D_0 \oplus (04D_2) = 1E \oplus 3C \oplus 75 = 57.$$

The multiplicative inverse of `02` is `8E`, so D_1=`8E·57=A5`. Recompute `P=3C⊕A5⊕5A=C3`. Check `02·A5=57` and `04·5A=75` to verify Q. Multiplication is carryless and reduced modulo `0x11D`; it is not ordinary integer multiplication.

0+1 and 1+0 failure sets

0+1 stripes(A, B) and(C, D), then mirrors the stripes. 1+0 mirrors(A, B) and(C, D), then stripes the pairs. After A fails,0+1 depends on complete(C, D); 1+0 uses B and C/D. A second C loss breaks 0+1 but leaves 1+0 withB/D. A second B loss instead leaves 0+1's complete CD, but destroys 1+0's AB mirror. Neither layout tolerates every arbitrary two-device loss.

With independent per-device loss probability p over one interval and no repair, a two-device mirror survives $1-p^2$. Four-disk RAID5 survives$(1-p)^4+4p(1-p)^3$. Common power/firmware/environment and repair/rebuild violate those independence/no-repair assumptions. These are model probabilities, not measured lifetimes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Checksums, storage pools and reservations

The book's ZFS example stores child addresses/checksums in parent metadata and validates down the tree. A checksum inside the same corrupt block is not equivalent to a trusted parent's end-to-end check. Detection requires the stated corruption model; repair additionally requires a correct redundant source. A checksum is not malicious-authentication protection.

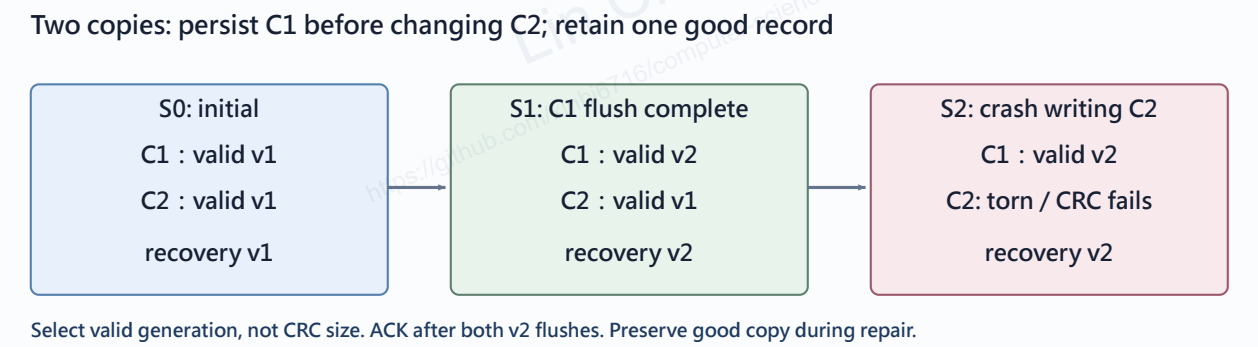
A storage pool supplies several filesystems; quotas limit growth, reservations protect shares. In an abstract 100 GiB pool, A uses 30, B20, free 50, A quota 60, B reservation 40 still needs 20 protected. Unreserved free=30, so A can grow min(quota remaining 30, unreserved 30)=30. Snapshots, dedup and compression are excluded; this is not all ZFS accounting or unbounded capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-copy stable storage

Ideal stable storage never loses information; implementations approximate it within a failure model. Fix serialized 512-byte records, two device copies, nonwrapping generation, CRC detecting the injected tears, and trustworthy durable flush. Write only one copy at a time. A crash/torn write affects the currently written copy; the untouched and previously flushed copies stay valid. Common loss of those copies is outside this invariant.

Record format is4-byte big-endian generation+504-byte payload+4-byte CRC32 over the first 508 bytes. V1 payload is ASCII V1 then 502 zeros, CRC BF4CA2E8; V2 similarly gives 4A112E80. Recovery validates CRC then selects highest generation, not the numerically largest CRC. CRC is neither collision-free arbitrary-corruption proof nor authentication.



Every crash prefix and the invariant

Write/flush C1, then write/flush C2, then ACK.

Crash prefix	C1	C2	Recoverable version
Initial	Valid v1	Valid v1	v1
During C1, no confirmed flush	v1/v2/torn	Valid v1	At least v1; complete valid v2 may win
C1 v2 flushed, before C2	Valid v2	Valid v1	v2
During C2	Valid v2	v1/v2/torn	v2, not v1
C2 v2 flushed, then ACK	Valid v2	Valid v2	Acknowledged v2

Induction starts with two valid copies. During C1 the untouched C2 is valid old; before C2 is modified, C1 is durable new. Hence at least one valid copy survives every prefix. Recovery preserves the selected good copy, repairs/flushes the other first, and still retains a good copy if repair crashes. Concurrently overwriting both, untrustworthy flush, or common loss breaks the premises.

An unacknowledged operation may recover old or complete new; no reply does not mean no execution. One-record recovery does not make a multi-record filesystem transaction atomic.

Flush, FUA, ordering and atomicity

Volatile device cache can complete before nonvolatile media. Flush asks specified prior writes to reach persistence; FUA makes that request's completion satisfy the durable contract. Native support, emulation, and propagation through layers follow driver/device rules. Modern Linux REQ_PREFLUSH/REQ_FUA are concrete examples, not every historical OSC 9 device's flag names.

CPU memory barriers are not device-cache flushes. Ordered writes can still tear. Two-copy record recovery is distinct from RAID5 parity consistency. The guarantee must connect host buffer, device cache, persistent media, and ACK under the same failure model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Positioning, redundancy and durable completion

HDD 53→14 versus 53→0 changes positioning. FTL maps L1 to a new page and moves live data. RAID reconstructs under a specified loss set. Stable-write ordering decides which version survives interrupted updates. Success in one layer cannot replace another layer's contract.

Keep queue wait, seek/rotation, transfer and completion scheduling separate. State request bytes with IOPS; host/physical bytes with WA; topology with RAID capacity; failure correlation/repair with reliability; flush/atomicity with ACK. Changing media, topology or cache strategy requires revisiting that layer's premises. Detection, repair sources and historical backups each serve a different need.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 10.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study